



Eduardo Dragone Pinheiro Corrêa(dragone.dev@gmail.com)

vehicle-rental-api

Sistema completo desenvolvido para locação e gerenciamento de veículos

Salvador

2026

Sumário

1. Entidades do Sistema

2. Diagramas do Sistema

2.1. Diagrama Entidade-Relacionamento (DER)

2.2. Diagrama de Classes UML — Customer

2.3. Diagrama de Classes UML — Vehicle

2.4. Diagrama de Classes UML — Employee

3. Stack de Desenvolvimento

4. Rotas do Sistema (*vehicle-rental-api*)

4.1. Rotas de Clientes (*CustomerController*)

- 4.1.1. POST /v1/customers
- 4.1.2. GET /v1/customers
- 4.1.3. GET /v1/customers/{id}
- 4.1.4. GET /v1/customers/document
- 4.1.5. PUT /v1/customers/{id}
- 4.1.6. DELETE /v1/customers/{id}
- 4.1.7. PATCH /v1/customers/{id}/reactivate
- 4.1.8. Endpoints — CustomerController

4.2. Rotas de Veículos (*VehicleController*)

- 4.2.1. POST /v1/vehicles
- 4.2.2. GET /v1/vehicles
- 4.2.3. GET /v1/vehicles/{id}
- 4.2.4. GET /v1/vehicles/status
- 4.2.5. PUT /v1/vehicles/{id}
- 4.2.6. PATCH /v1/vehicles/{id}/status
- 4.2.7. DELETE /v1/vehicles/{id}
- 4.2.8. PATCH /v1/vehicles/{id}/reactivate

- 4.2.9. Endpoints — VehicleController

4.3. Rotas de Funcionários (*EmployeeController*)

- 4.3.1. POST /v1/employees
- 4.3.2. GET /v1/employees
- 4.3.3. GET /v1/employees/{id}
- 4.3.4. GET /v1/employees/employeeCode
- 4.3.5. PUT /v1/employees/{id}
- 4.3.6. DELETE /v1/employees/{id}
- 4.3.7. PATCH /v1/employees/{id}/reactivate
- 4.3.8. Endpoints — EmployeeController

4.4. Rotas de Pedidos de Locação (*RentalOrderController*)

- 4.4.1. POST /v1/rental-orders — Criar pedido de locação
- 4.4.2. GET /v1/rental-orders — Listar pedidos de locação paginados
- 4.4.3. GET /v1/rental-orders/{id} — Buscar pedido de locação por ID
- 4.4.4. GET /v1/rental-orders/status — Buscar pedidos por status
- 4.4.5. PUT /v1/rental-orders/{id} — Atualizar pedido de locação
- 4.4.6. PATCH /v1/rental-orders/{id}/cancel — Cancelar pedido de locação
- 4.4.7. PATCH /v1/rental-orders/{id}/confirm — Confirmar pedido de locação
- 4.4.8. PATCH /v1/rental-orders/{id}/start — Iniciar locação
- 4.4.9. PATCH /v1/rental-orders/{id}/finish — Finalizar locação
- 4.4.10. PATCH /v1/rental-orders/{id}/pay — Registrar pagamento
- 4.4.11. Endpoints — RentalOrderController

5. Introdução

5.1. Objetivo do Sistema

5.2. Breve descrição do vehicle-rental-api

5.3. Processo de Locação

5.4. Problema que a API resolve

5.5. Benefícios em Cenário Real

1. Entidades do Sistema

1- Cliente (Locatário)

Descrição: Representa o cliente responsável por realizar as locações de veículos no sistema. Armazena informações cadastrais e de contato necessárias para identificação e realização das reservas.

2- Funcionário (Agente de Locação)

Descrição: Representa o funcionário responsável pelo gerenciamento e faturamento das locações de veículos no sistema. Armazena informações cadastrais e de contato necessárias para sua identificação e para o controle das operações realizadas dentro da empresa.

3- Veículo

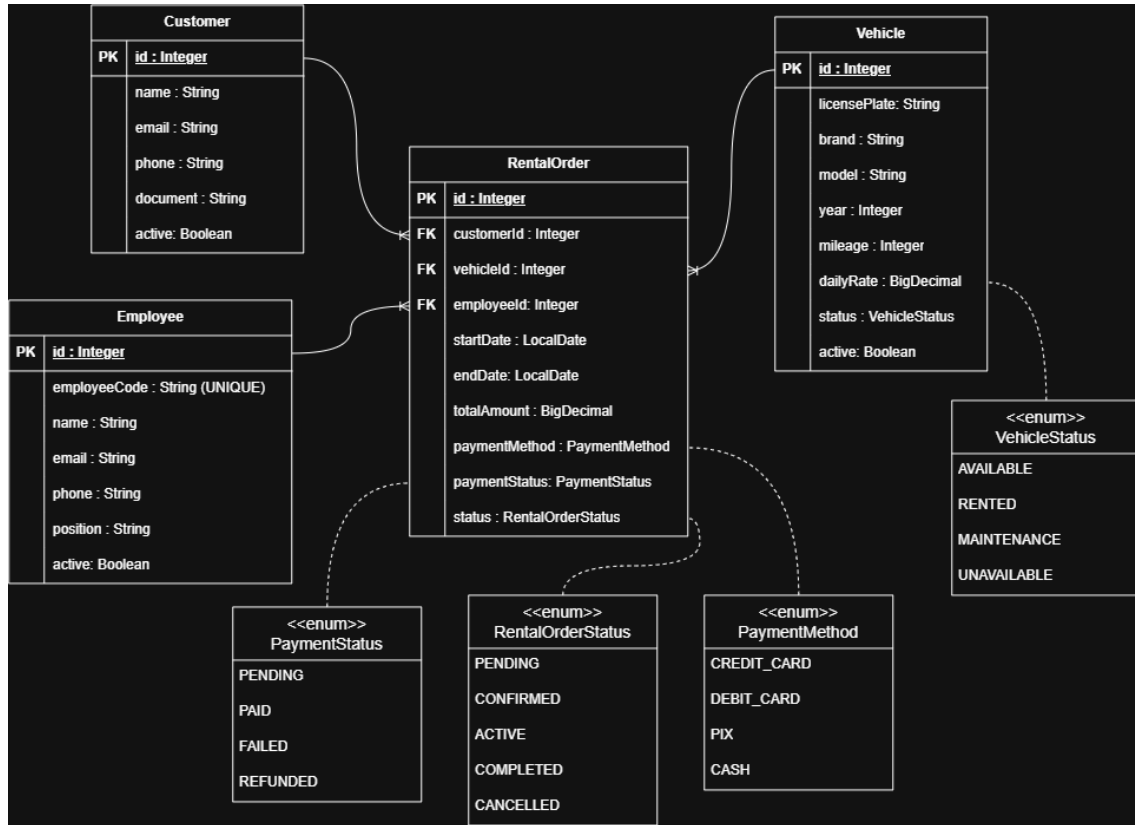
Descrição: Representa o veículo disponibilizado pela empresa para locação. Armazena informações necessárias para sua identificação, caracterização, disponibilidade e gerenciamento dentro do sistema.

4- Pedido (RentalOrder)

Descrição: Representa o pedido de locação de um veículo realizado pelo cliente. Armazena todas as informações relacionadas à operação, incluindo o cliente, veículo, funcionário responsável, período da locação, valores, dados de pagamento, condições contratuais e status do pedido.

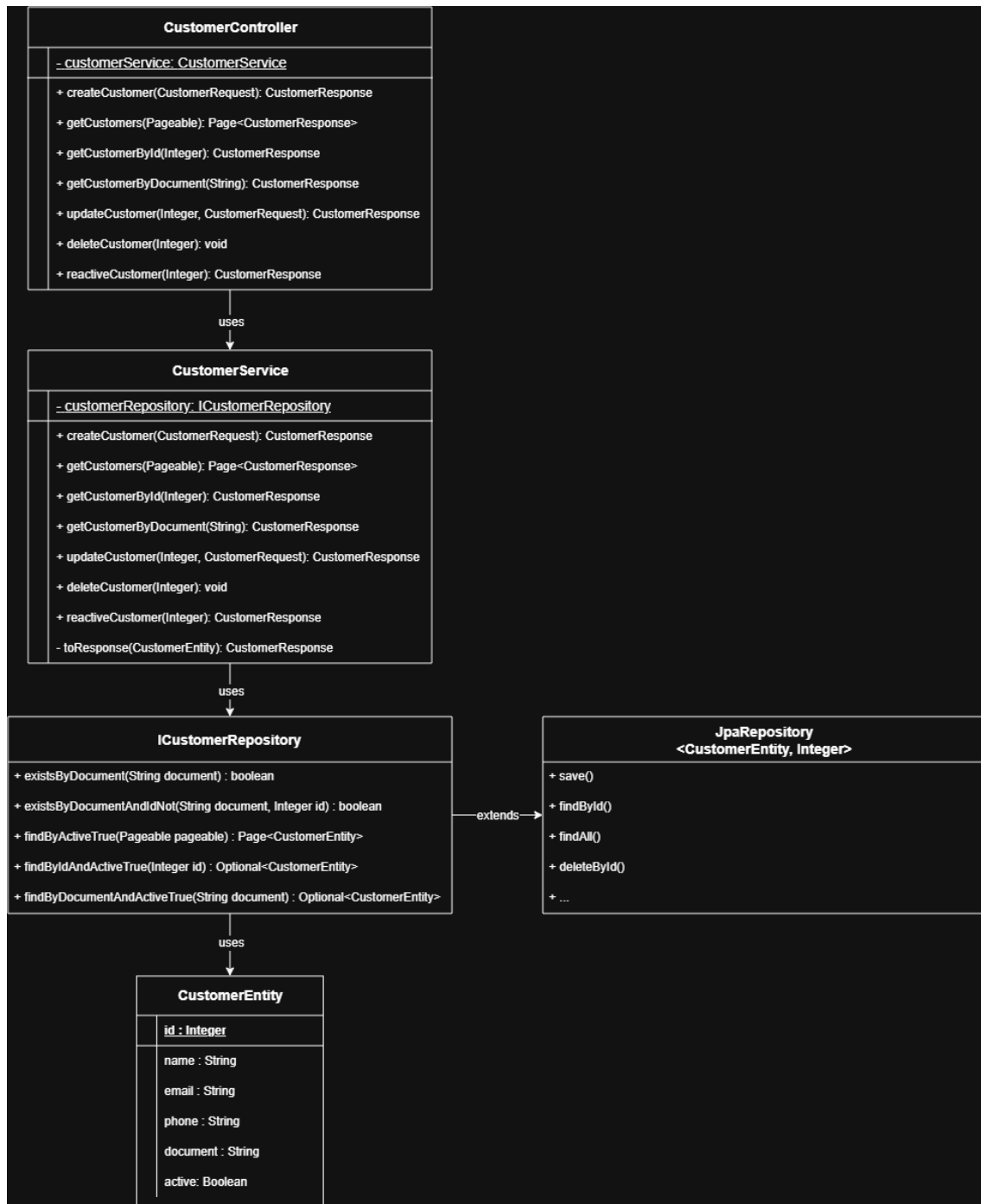
2. Diagramas do Sistema

2.1. Diagrama Entidade-Relacionamento (DER)



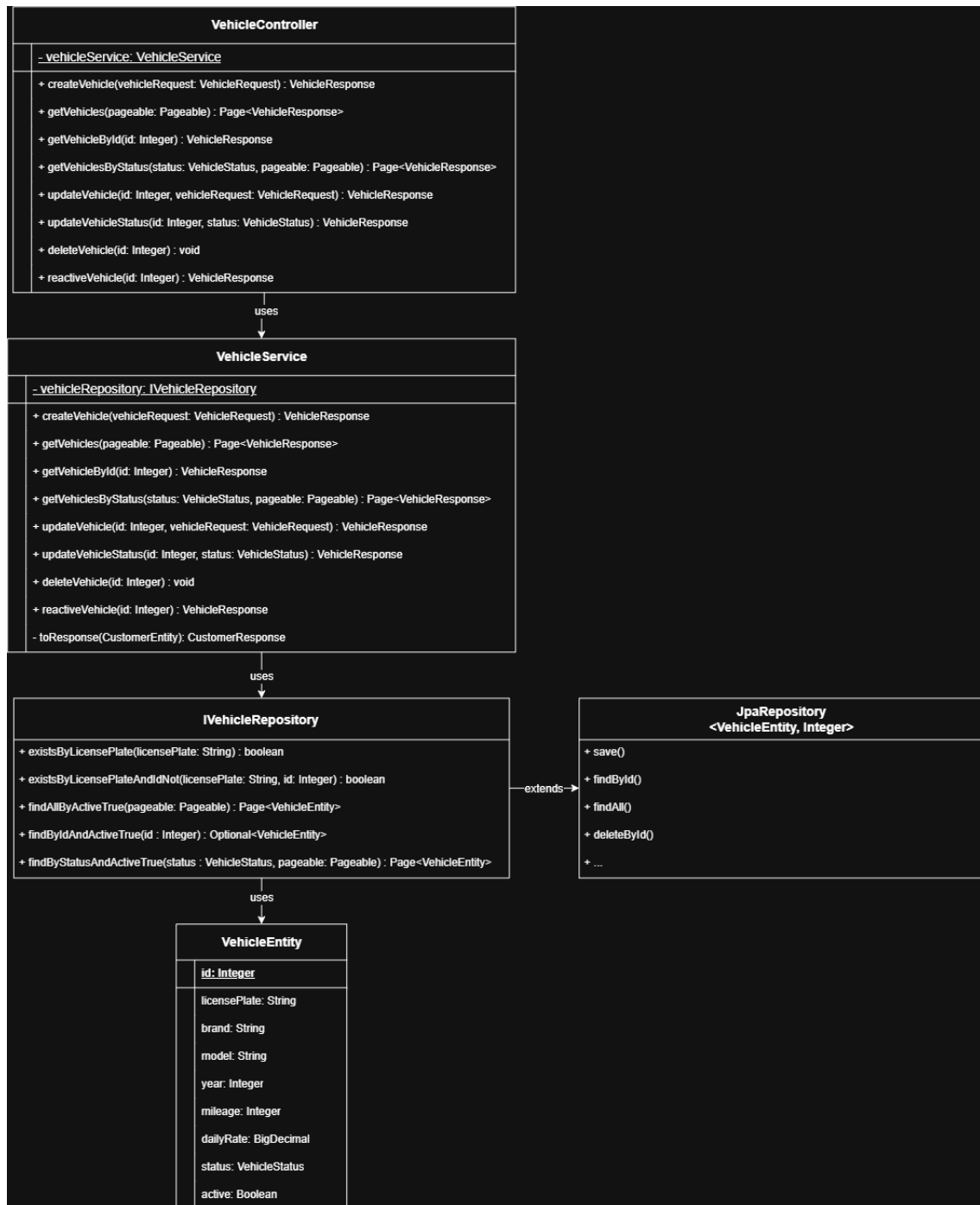
Descrição: Representação gráfica da estrutura do banco de dados do sistema, apresentando suas entidades, atributos, chaves e relacionamentos. O diagrama permite visualizar como os dados do sistema de locação de veículos são organizados e relacionados entre si.

2.2. Diagrama de Classes UML — Customer



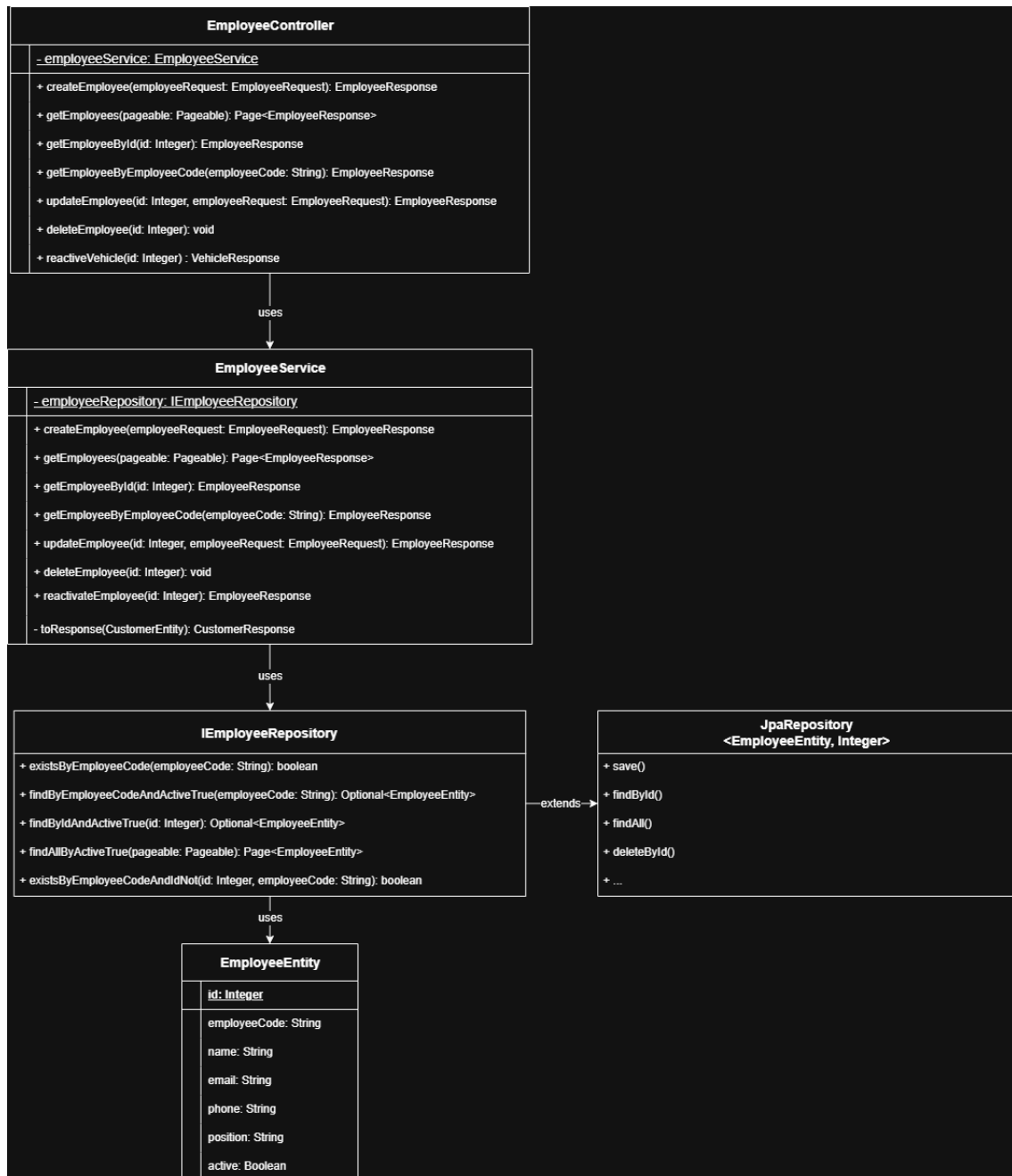
Descrição: Representação gráfica da estrutura da classe Customer, apresentando seus atributos, métodos e características principais. O diagrama permite visualizar a organização da entidade responsável pelo gerenciamento dos clientes no sistema de locação de veículos.

2.2. Diagrama de Classes UML — Vehicle



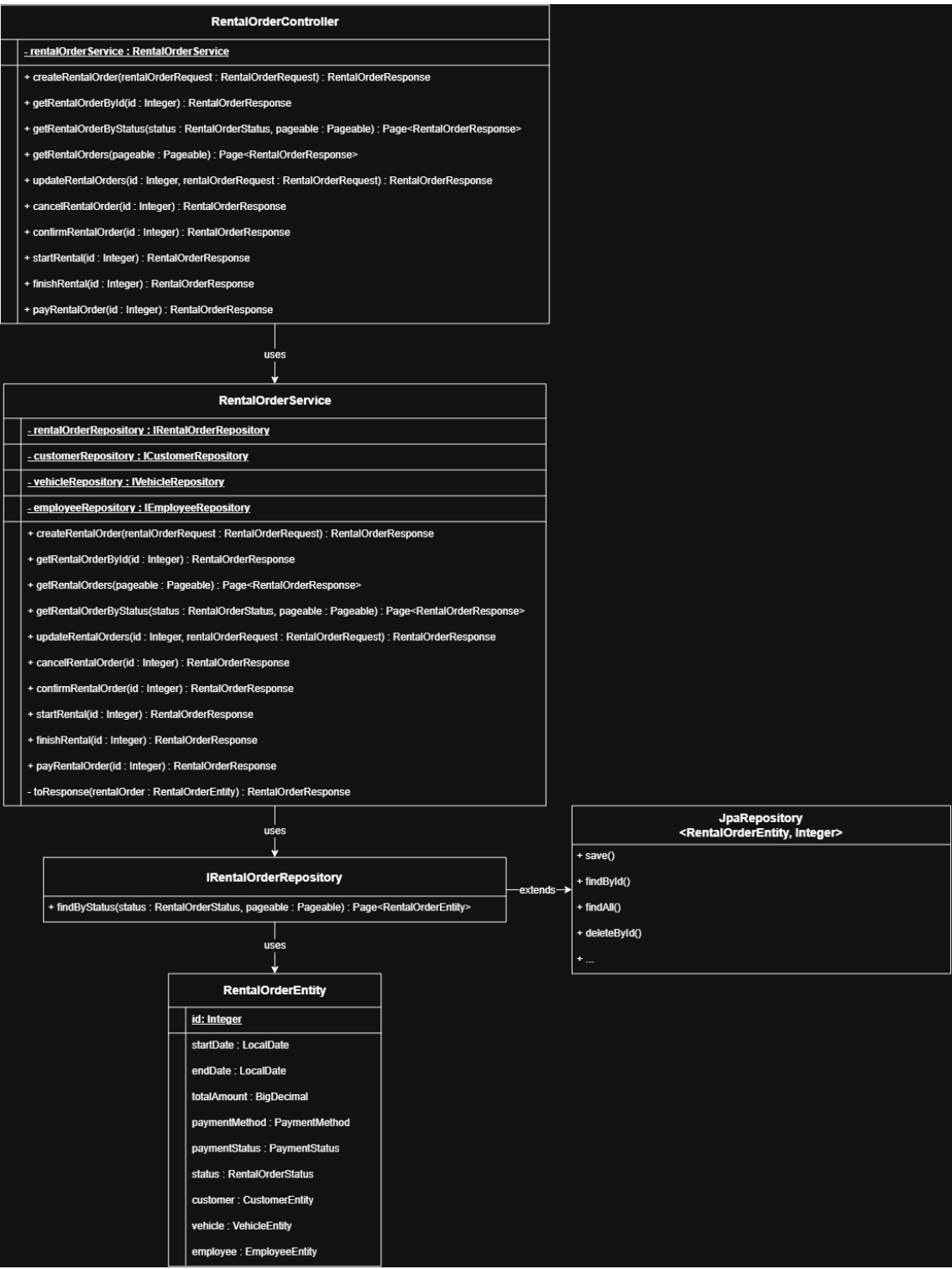
Descrição: Representação gráfica da estrutura da classe Vehicle, apresentando seus atributos, métodos e características principais. O diagrama permite visualizar a organização da entidade responsável pelo gerenciamento dos veículos no sistema de locação.

2.3. Diagrama de Classes UML — Employee



Descrição: Representação gráfica da estrutura da classe Employee, apresentando seus atributos, métodos e características principais. O diagrama permite visualizar a organização da entidade responsável pelo gerenciamento dos funcionários no sistema de locação.

2.4. Diagrama de Classes UML — RentalOrder



Descrição: Representação gráfica da estrutura da classe RentalOrder, apresentando seus atributos, métodos e características principais. O diagrama

permite visualizar a organização da entidade responsável pelo gerenciamento dos pedidos de locação de veículos no sistema.

3. Stack de Desenvolvimento

1- Linguagem de programação: Java 21

2- Framework: Spring Boot

2.1- Dependências: Spring Web, Spring Data JPA, Lombok, PostgreSQL Driver, Spring Boot Validation

3- Banco de dados: PostgreSQL

4- Ferramentas de Desenvolvimento: Maven, Git, GitHub, IntelliJ IDEA, pgAdmin 4

5- Ferramentas de Gerenciamento: Trello

4. Rotas do Sistema (*vehicle-rental-api*)

4.1. Rotas de Clientes (*CustomerController*)

4.1.1. POST /v1/customers

Descrição: Cria um novo cliente.

Request Body:

```
{
  "name": "eduardo montgomery",
  "email": "eduardo.montgomery@email.com",
  "phone": "71988888888",
  "document": "74629381504"
}
```

Response: 201 Created

4.1.2. GET /v1/customers

Descrição: Retorna os clientes de forma paginada.

Query Parameters:

Parâmetro	Tipo	Descrição
page	Integer	Número da página
size	Integer	Quantidade de registros por página
sort	String	Campo e direção da ordenação

Exemplo: GET /v1/customers?page=0&size=2&sort=id,asc

Response:

```
{
  "content": [],

```

```
"totalElements": 6,  
"totalPages": 3,  
"size": 2,  
"number": 0  
}
```

Response: 200 OK

4.1.3. GET /v1/customers/{id}

Descrição: Retorna um cliente pelo ID.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do cliente

Exemplo: GET /v1/customers/1

Response: 200 OK

4.1.4. GET /v1/customers/document

Descrição: Retorna um cliente pelo documento.

Query Parameter:

Parâmetro	Tipo	Obrigatório	Descrição
document	String	Sim	Documento do cliente

Exemplo: GET /v1/customers/document?document=74629381504

Response: 200 OK

4.1.5. PUT /v1/customers/{id}

Descrição: Atualiza os dados de um cliente.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do cliente

Request Body:

```
{  
  "name": "eduardo montgomery",  
  "email": "eduardo.novo@email.com",  
  "phone": "71988888888",  
  "document": "74629381504"  
}
```

Response: 200 OK

Atualiza os dados do cliente. Por utilizar o método PUT, a requisição deve fornecer todos os dados do recurso, mesmo quando apenas um campo será alterado.

4.1.6. DELETE /v1/customers/{id}

Descrição: Remove logicamente o cliente.

Realiza a remoção lógica do cliente, mantendo o registro no banco de dados e alterando seu atributo active para false.

Exemplo: DELETE /v1/customers/1

Response: 204 No Content

4.1.7. PATCH /v1/customers/{id}/reactive

Descrição: Reativa um cliente que está inativo, alterando seu atributo active para true.

Exemplo: PATCH /v1/customers/1/reactive

Request Body: Nenhum.

Response: 200 OK

4.1.8. Endpoints — CustomerController

Método	Endpoint	Função
POST	/v1/customers	Criar cliente
GET	/v1/customers	Listar clientes paginados
GET	/v1/customers/{id}	Buscar cliente
GET	/v1/customers/document	Buscar cliente por documento
PUT	/v1/customers/{id}	Atualizar cliente
DELETE	/v1/customers/{id}	Desativar cliente
PATCH	/v1/customers/{id}/reactive	Reativar cliente

4.2. Rotas de Veículos (VehicleController)

4.2.1. POST /v1/vehicles

Descrição: Cria um novo veículo.

Request Body:

```
{  
  "licensePlate": "ABC1D23",  
  "brand": "Porsche",  
  "model": "911 Carrera",  
}
```

```
"year": 2025,  
"mileage": 5000,  
"dailyRate": 1299.90  
}
```

Response: 201 Created

4.2.2. GET /v1/vehicles

Descrição: Retorna os veículos de forma paginada.

Query Parameters:

Parâmetro	Tipo	Descrição
page	Integer	Número da página
size	Integer	Quantidade de registros por página
sort	String	Campo e direção da ordenação

Exemplo: GET /v1/vehicles?page=0&size=2&sort=id,asc

```
{  
  "content": [],  
  "totalElements": 10,  
  "totalPages": 5,  
  "size": 2,  
  "number": 0  
}
```

Response: 200 OK

4.2.3. GET /v1/vehicles/{id}

Descrição: Retorna um veículo pelo ID.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do veículo

Exemplo: GET /v1/vehicles/1

Response: 200 OK

4.2.4. GET /v1/vehicles/status

Descrição: Retorna veículos filtrados por status, de forma paginada.

Query Parameters:

Parâmetro	Tipo	Obrigatório
status	VehicleStatus	Sim
page	Integer	Não
size	Integer	Não

Status disponíveis:

AVAILABLE

RENTED

MAINTENANCE

UNAVAILABLE

Exemplo: GET /v1/vehicles/status?status=AVAILABLE&page=0&size=2

Response: 200 OK

4.2.5. PUT /v1/vehicles/{id}

Descrição: Atualiza os dados de um veículo.

Path Parameter: id: Integer

Request Body:

```
{  
  "licensePlate": "ABC1D23",  
  "brand": "Porsche",  
  "model": "911 Carrera",  
  "year": 2025,  
  "mileage": 7500,  
  "dailyRate": 1399.90  
}
```

Response: 200 OK

Atualiza os dados do veículo. Por utilizar o método PUT, a requisição deve fornecer todos os dados do recurso, mesmo quando apenas um campo será alterado.

4.2.6. PATCH /v1/vehicles/{id}/status

Descrição: Atualiza somente o status do veículo.

Exemplo: PATCH /v1/vehicles/1/status?status=MAINTENANCE

Status disponíveis:

AVAILABLE

RENTED

MAINTENANCE

UNAVAILABLE

Response: 200 OK

4.2.7. DELETE /v1/vehicles/{id}

Descrição: Desativa logicamente o veículo, alterando seu atributo active para false e mantendo o registro no banco de dados.

Exemplo: DELETE /v1/vehicles/1

Response: 204 No Content

4.2.8. PATCH /v1/vehicles/{id}/reactive

Descrição: Reativa um veículo que foi desativado logicamente, alterando seu atributo active para true e tornando-o novamente disponível no sistema.

Exemplo: PATCH /v1/vehicles/1/reactive

Request Body: Nenhum.

Response: 200 OK

4.2.9. Endpoints — VehicleController

Método	Endpoint	Função
POST	/v1/vehicles	Criar veículo
GET	/v1/vehicles	Listar veículos paginados
GET	/v1/vehicles/{id}	Buscar veículo
GET	/v1/vehicles/status	Filtrar por status
PUT	/v1/vehicles/{id}	Atualizar veículo
PATCH	/v1/vehicles/{id}/status	Alterar status
DELETE	/v1/vehicles/{id}	Desativar veículo
PATCH	/v1/vehicles/{id}/reactive	Reativar veículo

4.3. Rotas de Funcionários (EmployeeController)

4.3.1. POST /v1/employees

Descrição: Cria um novo funcionário.

Request Body:

```
{  
  "employeeCode": "EMP001",  
  "name": "Carlos Silva",  
  "email": "carlos.silva@email.com",  
  "phone": "71999999999",  
  "position": "Manager"  
}
```

Response: 201 Created

4.3.2. GET /v1/employees

Descrição: Retorna os funcionários ativos de forma paginada.

Query Parameters:

Parâmetro	Tipo	Descrição
page	Integer	Número da página
size	Integer	Quantidade de registros por página
sort	String	Campo e direção da ordenação

Exemplo: GET /v1/employees?page=0&size=2&sort=id,asc

Response:

```
{  
  "content": [],  
  "totalElements": 10,  
}
```

```
"totalPages": 5,  
"size": 2,  
"number": 0  
}
```

Response: 200 OK

4.3.3. GET /v1/employees/{id}

Descrição: Retorna um funcionário ativo pelo ID.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do funcionário

Exemplo: GET /v1/employees/1

Response: 200 OK

4.3.4. GET /v1/employees/employeeCode

Descrição: Retorna um funcionário ativo pelo seu código de identificação.

Query Parameter:

Parâmetro	Tipo	Obrigatório	Descrição
employeeCode	String	Sim	Código identificador do funcionário

Exemplo: GET /v1/employees/employeeCode?employeeCode=EMP001

Response: 200 OK

4.3.5. PUT /v1/employees/{id}

Descrição: Atualiza os dados de um funcionário.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do funcionário

Request Body:

```
{  
  "employeeCode": "EMP001",  
  "name": "Carlos Silva",  
  "email": "carlos.silva@email.com",  
  "phone": "719888888888",  
  "position": "Administrator"  
}
```

Response: 200 OK

Atualiza os dados do funcionário. Por utilizar o método PUT, a requisição deve fornecer todos os dados do recurso.

4.3.6. DELETE /v1/employees/{id}

Descrição: Desativa logicamente um funcionário.

A remoção é realizada por **soft delete**, mantendo o registro no banco de dados e alterando o atributo active para false.

Exemplo: DELETE /v1/employees/1

Response: 204 No Content

4.3.7. PATCH /v1/employees/{id}/reactivate

Descrição: Reativa um funcionário que foi desativado, alterando seu atributo active para true.

Exemplo: PATCH /v1/employees/1/reactivate

Request Body: Nenhum.

Response: 200 OK

4.3.8. Endpoints — EmployeeController

Método	Endpoint	Função
POST	/v1/employees	Criar funcionário
GET	/v1/employees	Listar funcionários paginados
GET	/v1/employees/{id}	Buscar funcionário por ID
GET	/v1/employees/employeeCode	Buscar funcionário por código
PUT	/v1/employees/{id}	Atualizar funcionário
DELETE	/v1/employees/{id}	Desativar funcionário
PATCH	/v1/employees/{id}/reactivate	Reativar funcionário

4.4. Rotas de Pedidos de Locação (RentalOrderController)

4.4.1. POST /v1/rental-orders

Descrição: Cria um novo pedido de locação.

Request Body:

```
{  
  "startDate": "2026-08-26",  
  "endDate": "2026-08-30",  
  "paymentMethod": "CREDIT_CARD",
```

```
"customerId": 1,  
"vehicleId": 1,  
"employeeId": 1  
}
```

Response: 201 Created

4.2.2. GET /v1/rental-orders

Descrição: Retorna os pedidos de locação de forma paginada.

Query Parameters:

Parâmetro	Tipo	Descrição
page	Integer	Número da página
size	Integer	Quantidade de registros por página
sort	String	Campo e direção da ordenação

Exemplo: GET /v1/rental-orders?page=0&size=2&sort=id,asc

Response:

```
{  
  "content": [],  
  "totalElements": 6,  
  "totalPages": 3,  
  "size": 2,  
  "number": 0  
}
```

Response: 200 OK

4.2.3. GET /v1/rental-orders/{id}

Descrição: Retorna um pedido de locação pelo ID.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do pedido de locação

Exemplo: GET /v1/rental-orders/1

Response: 200 OK

4.2.4. GET /v1/rental-orders/status

Descrição: Retorna os pedidos de locação filtrados por status, de forma paginada.

Query Parameters:

Parâmetro	Tipo	Obrigatório	Descrição
status	RentalOrderStatus	Sim	Status do pedido de locação
page	Integer	Não	Número da página
size	Integer	Não	Quantidade de registros por página
sort	String	Não	Campo e direção da ordenação

Exemplo: GET /v1/rental-orders/status?status=CONFIRMED&page=0&size=10

Response: 200 OK

4.2.5. PUT /v1/rental-orders/{id}

Descrição: Atualiza os dados de um pedido de locação.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do pedido de locação

Request Body:

```
{  
  "startDate": "2026-08-27",  
  "endDate": "2026-08-31",  
  "paymentMethod": "PIX",  
  "customerId": 1,  
  "vehicleId": 1,  
  "employeeId": 1  
}
```

Response: 200 OK

Atualiza as informações do pedido e recalcula o valor total da locação com base no período informado e na diária do veículo.

4.2.6. PATCH /v1/rental-orders/{id}/cancel

Descrição: Cancela um pedido de locação.

O cancelamento só pode ser realizado quando o pedido estiver com status CONFIRMED.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do pedido de locação

Exemplo: PATCH /v1/rental-orders/1/cancel

Request Body: Nenhum.

Response: 200 OK

4.2.7. PATCH /v1/rental-orders/{id}/confirm

Descrição: Confirma um pedido de locação.

A confirmação só pode ser realizada quando o pedido estiver com status PENDING.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do pedido de locação

Exemplo: PATCH /v1/rental-orders/1/confirm

Request Body: Nenhum.

Response: 200 OK

4.2.8. PATCH /v1/rental-orders/{id}/start

Descrição: Inicia a locação de um veículo.

O pedido deve estar com status CONFIRMED e o veículo deve estar disponível. Ao iniciar a locação, o pedido passa para ACTIVE e o veículo passa para RENTED.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do pedido de locação

Exemplo: PATCH /v1/rental-orders/1/start

Request Body: Nenhum.

Response: 200 OK

4.2.9. PATCH /v1/rental-orders/{id}/finish

Descrição: Finaliza uma locação ativa.

O pedido deve estar com status ACTIVE. Ao finalizar a locação, o pedido passa para COMPLETED e o veículo retorna para o status AVAILABLE.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do pedido de locação

Exemplo: PATCH /v1/rental-orders/1/finish

Request Body: Nenhum.

Response: 200 OK

4.2.10. PATCH /v1/rental-orders/{id}/pay

Descrição: Registra o pagamento de um pedido de locação.

O status de pagamento é alterado de PENDING para PAID. Caso o pedido já esteja pago, uma operação não permitida é retornada.

Path Parameter:

Parâmetro	Tipo	Descrição
id	Integer	Identificador do pedido de locação

Exemplo: PATCH /v1/rental-orders/1/pay

Request Body: Nenhum.

Response: 200 OK

4.2.11. Endpoints — RentalOrderController

Método	Endpoint	Função
POST	/v1/rental-orders	Criar pedido de locação
GET	/v1/rental-orders	Listar pedidos de locação paginados
GET	/v1/rental-orders/{id}	Buscar pedido de locação por ID
GET	/v1/rental-orders/status	Buscar pedidos por status
PUT	/v1/rental-orders/{id}	Atualizar pedido de locação
PATCH	/v1/rental-orders/{id}/cancel	Cancelar pedido de locação
PATCH	/v1/rental-orders/{id}/confirm	Confirmar pedido de locação
PATCH	/v1/rental-orders/{id}/start	Iniciar locação
PATCH	/v1/rental-orders/{id}/finish	Finalizar locação
PATCH	/v1/rental-orders/{id}/pay	Registrar pagamento

5. Introdução

5.1. Objetivo do Sistema

O **Vehicle Rental API** tem como objetivo fornecer uma solução para o **gerenciamento de processos de locação de veículos**, permitindo controlar clientes, funcionários, veículos e pedidos de locação de forma centralizada e organizada.

A API foi desenvolvida utilizando **Java e Spring Boot**, seguindo uma arquitetura baseada na separação de responsabilidades entre as diferentes camadas da aplicação. O sistema permite realizar operações de cadastro, consulta, atualização e gerenciamento dos diferentes recursos envolvidos no processo de locação.

Além das operações básicas de CRUD, o sistema possui **regras de negócio** para controlar o ciclo de uma locação, como confirmação, início, finalização, cancelamento e registro do pagamento.

5.2. Breve descrição do vehicle-rental-api

O vehicle-rental-api é uma **API REST para gerenciamento de locação de veículos**, desenvolvida com o objetivo de representar um cenário próximo ao encontrado em empresas reais do setor de aluguel de automóveis.

O sistema é composto principalmente pelas seguintes entidades:

- **Customer:** representa os clientes que realizam as locações.
- **Vehicle:** representa os veículos disponíveis para locação e seus respectivos estados.
- **Employee:** representa os funcionários responsáveis pela operação do sistema.
- **RentalOrder:** representa o pedido de locação, relacionando cliente, veículo e funcionário, além de armazenar informações como período da locação, valor total, método de pagamento e status do pedido.

5.3. Processo de Locação

O processo de locação segue um fluxo controlado pela aplicação:



Também é possível cancelar um pedido durante o fluxo permitido:



Dessa forma, a API não apenas armazena informações, mas também **controla as operações que podem ou não ser realizadas em cada etapa da locação**.

5.4. Problema que a API resolve

Em uma empresa de locação de veículos, controlar manualmente informações sobre **clientes, veículos, funcionários, pagamentos e períodos de locação** pode gerar diversos problemas. A ausência de um sistema centralizado pode resultar em informações desatualizadas, conflitos na disponibilidade dos veículos, erros no cálculo do valor da locação e dificuldade para acompanhar o andamento dos pedidos.

Um exemplo de situação real seria uma empresa possuir um veículo disponível e, simultaneamente, dois funcionários tentarem realizar uma locação para o mesmo automóvel. Sem um controle adequado, poderiam ser registrados dois pedidos para o mesmo veículo, causando um conflito operacional.

O vehicle-rental-api busca solucionar esse tipo de problema através de **regras de negócio implementadas na própria aplicação**. Antes de criar uma locação, por

exemplo, o sistema verifica se o veículo está disponível. Durante o início da locação, o veículo passa para o status RENTED, impedindo que seja tratado como disponível para uma nova operação.

Outro exemplo está no cálculo do valor da locação. Em vez de depender de um cálculo realizado manualmente por um funcionário, a API calcula o valor total com base no **período da locação e na diária do veículo**, reduzindo a possibilidade de erros.

Assim, a solução pode ser utilizada como base para um sistema real de uma **locadora de veículos**, permitindo futuramente a integração com uma aplicação web ou mobile utilizada pelos funcionários e clientes.

5.5. Benefícios em cenário real

- Centralização das informações de clientes, veículos, funcionários e locações;
- Controle da disponibilidade dos veículos;
- Redução de erros manuais no processo de locação;
- Cálculo automático do valor total da locação;
- Controle do ciclo de vida dos pedidos;
- Registro e acompanhamento dos pagamentos;
- Facilidade de integração com sistemas web, mobile ou outros serviços;
- Maior organização e rastreabilidade das operações realizadas.

Em uma aplicação real, essa API poderia funcionar como o back-end de uma locadora de veículos, fornecendo os serviços necessários para que um sistema web ou aplicativo permita consultar veículos disponíveis, realizar reservas, acompanhar locações, controlar pagamentos e administrar os recursos da empresa.